Name: _____ Student NUID: _____

CS 2100: Program Design and Implementation 1

Practice Quiz 4

Instructions

- Do not begin the quiz until instructed to do so.
- You may use both sides of a sheet of paper up to 8.5"x11" for reference, but no other resources, including phones, computers, AI, headphones, and ear pods.
- You have until the end of the class period to complete the quiz.
- Students may not leave the classroom during the first 10 minutes of the quiz (except in case of emergency).
- Hand your completed quiz to an instructor before leaving the room.
- Talk to an instructor if you need to leave the room and reenter.

1. Stakeholder-value matrices (revisited from Quiz 2)

Consider an event ticket purchasing online platform (like Eventbrite, Ticketmaster, or SeatGeek). Event hosts can advertise tickets to be purchased on this platform.

Please complete the following stakeholder-value matrix. You only need to fill out the two empty cells in the bottom row.

	Amount of personal information shared by customers	**Number of tickets that can be purchased at the same time**
Customer / event attendee	Likely wants to provide minimal amount of personal information	Likely doesn't care, as long as the limit is high enough for their needs
Event host		

2. Correlation (revisited from Quiz 2)

Let's say you're hosting a party. There's a long line of people waiting to enter. Also, as people leave, you ask them how much they enjoyed the party (on a scale of 1 - 5).

You want to know whether wait times (in minutes) are correlated with attendees' enjoyment ratings. So, you perform the following calculations:

```
wait_times = [1, 3.4, 15, 7, 14] # number of minutes each attendee waited
enjoyment = [5, 4.5, 1, 2.5, 1.3] # attendees' enjoyment ratings

print(np.corrcoef(wait_times, enjoyment))
```

The result is:

```
[[ 1.         -0.97780814]
 [-0.97780814  1.        ]]
```

Which of the following can you conclude from this data? Please circle all that are true.

A. Wait times are positively correlated with enjoyment.
B. Wait times are negatively correlated with enjoyment.
C. Waiting a long time causes attendees to enjoy the party more.
D. Waiting a long time causes attendees to dislike the party more.
E. Enjoying the party causes other attendees to wait in line longer.
F. Party-goers who are less interested "give up" when they see the long lines, so they don't attend the party.
G. Attendees who are highly interested in the party often convince the lines to move faster.

3. Properties (revisited from Quiz 3)

Consider the following class that represents an event attendee with an email address. Please add a setter for the `email` property that updates the `self._email` attribute if provided a valid email, and raises a `ValueError` if provided an invalid email.

An email is valid if it contains `'@'`.

Documentation is not required or graded on this question.

```
class Attendee:
    """An event attendee"""

    def __init__(self, email: str):
        """Initialize the attendee.

        Args:
            email (str): the attendee's email address
        """
        self.email = email

    @property
    def email(self) -> str:
        """Return the attendee's email address"""
        return self._email
```

4. Inheritance and abstract methods (revisited from Quiz 3)

The following `Attendee` abstract class represents attendees at a party. It has two concrete subclasses, representing types of attendees: `Guest` and `Volunteer`.

Your task is to complete the `Attendee` abstract class by giving it an abstract method `contribution()`. `Attendee` should be impossible to instantiate, but `Guest` and `Volunteer` should be possible to instantiate.

Documentation is not required or graded for this question.

```
class Attendee(ABC):
    """An event attendee"""

    def __init__(self, email: str):
        """
        Initialize the event attendee.
```

```
        Args:
            email (str): The attendee's email address
        """
        self._email = email
```

```
class Guest(Attendee):
    """A guest to the event"""
    def __init__(self, email: str, gift: bool):
        """Initialize the guest.

        Args:
            email (str): the guest's email address
            gift (bool): whether or not the guest is bringing a gift
        """
        super().__init__(email)
        self._gift = gift

    def contribution(self) -> str:
        if self._gift:
            return "Bringing a gift"
        else:
            return "Bringing themselves"

class Volunteer(Attendee):
    def __init__(self, email: str, role: str):
        super().__init__(email)
        self._role = role

    def contribution(self) -> str:
        return f'Volunteering as the {self._role}'
```

5. Coupling / cohesion / encapsulation (revisited from Quiz 3)

Please select all statements which are true.

A. If changing one class frequently requires changes to another class, those two classes are likely tightly coupled.
B. A function that only depends on its parameters and returns a value (with no other side effects like printing or updating global variables) must be highly cohesive.
C. High cohesion means that all the methods and attributes in a class are closely relate to each other.
D. Prefixing an attribute with a double underscore (`__`) makes it completely inaccessible from outside the class.
E. A class whose instance variables are all named without underscores (i.e., `self.name` instead of `self._name`) is well-encapsulated.

---

6. Iterator and Comparable

The code below models a playlist of songs at a party. Party attendees vote for songs to be played, and songs with the most votes are played sooner.

Songs in the playlist are sorted by votes (more votes = higher priority), with name as a tiebreaker (alphabetically earlier = higher priority).

Please add the methods `__lt__()` in `Song` and `__iter__()` in `Playlist`.

`__lt__()` should return `True` if `self` has higher priority than `other` (so a sorted list has the highest priority first).

`__iter__()` should return an `Iterator` that iterates through the songs in order of priority.

Documentation is not required or graded for this question.

```
class Song:
    """A song"""

    def __init__(self, name: str):
        """
        Initialize the song.

        Args:
            name (str): the attendee's name
        """
        self._name = name
        self._votes = 0

    def add_vote(self) -> None:
        """Add a vote for this song"""
        self._votes += 1

    @property
    def name(self) -> str:
        return self._name

    @property
    def votes(self) -> int:
        return self._votes

    def __repr__(self) -> str:
        """Return a string with the name and number of votes"""
        return f"{self._name} ({self._votes} votes)"

    def __eq__(self, other: object) -> bool:
        """Return True if other is a Song with the same name and votes as self,
        and False otherwise"""
        if isinstance(other, Song):
            return self._name == other.name and \
```

```
                    self._votes == other.votes
        return False
```

Put `__lt__()` here.

```
class Playlist(Iterable[Song]):
    """A playlist of songs"""

    def __init__(self, songs: list[Song]):
        """Initialize the playlist.

        Args:
            songs (list[Song]): a list of songs
        """
        self._sorted = sorted(songs)
        self._index = 0
```

Put `__iter__()` here.

7. Decorator / Strategy / Observer / Data Pull

Please circle all that are true.

A. After applying a function decorator, the original function's code is permanently deleted and can no longer be executed.
B. The Strategy pattern defines a family of interchangeable algorithms and lets the client choose which one to use at runtime.
C. Passing a sorting function as a parameter to sorted() (e.g. sorted(items, key=fn)) is an example of the Strategy pattern.
D. In the Observer pattern, the subject (publisher) maintains a list of observers and notifies them when its state changes.
E. In the Data Pull pattern, a client proactively requests data from a source rather than waiting to be notified.